

Capítulo 1 — La primera hora

Prolog nació en Marsella

Alain Colmerauer (1941–2017)

Prolog: Marsella, 1972

Foto: Alaindavid2, CC BY-SA 4.0,
Wikimedia Commons



El recorrido



Un programa

% padre(P, H): P es el padre de H.

padre(juan, ana).

padre(juan, pedro).

padre(pedro, luis).

padre(pedro, eva).

%! abuelo(?A, ?N) is nondet.

%

% A es abuelo de N cuando es el padre de su padre.

abuelo(A, N) :-

 padre(A, P),

 padre(P, N).

SWISH

swish.swi-prolog.org

```
padre(juan, ana).  
padre(juan, pedro).  
padre(pedro, luis).  
padre(pedro, eva).  
  
abuelo(A, N) :-  
    padre(A, P),  
    padre(P, N).
```

1 · el programa

```
?- padre(juan, ana).  
true.
```

3 · la respuesta

```
?- padre(juan, ana).
```

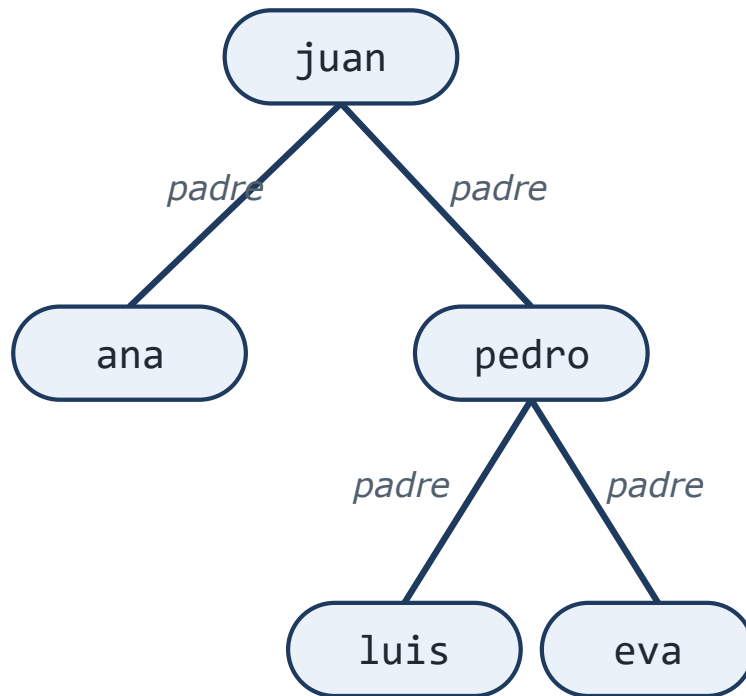
2 · la consulta

Run!

true. y false.

?- padre(juan, ana).
true.

?- padre(juan, luis).
false.



Un hecho

predicado
padre(**juan, ana**)**.**
dos argumentos
punto final

nombre y aridad: padre/2

Los hechos de padre/2

% padre(P, H): P es el padre de H.

padre(juan, ana).

padre(juan, pedro).

padre(pedro, luis).

padre(pedro, eva).

?- padre(juan, Quien).

Quien = ana ;

Quien = pedro.

Una regla

cabeza

lo que se concluye

abuelo(A, N)

: -

cuerpo

lo que debe cumplirse

padre(A, P) , padre(P, N)

si

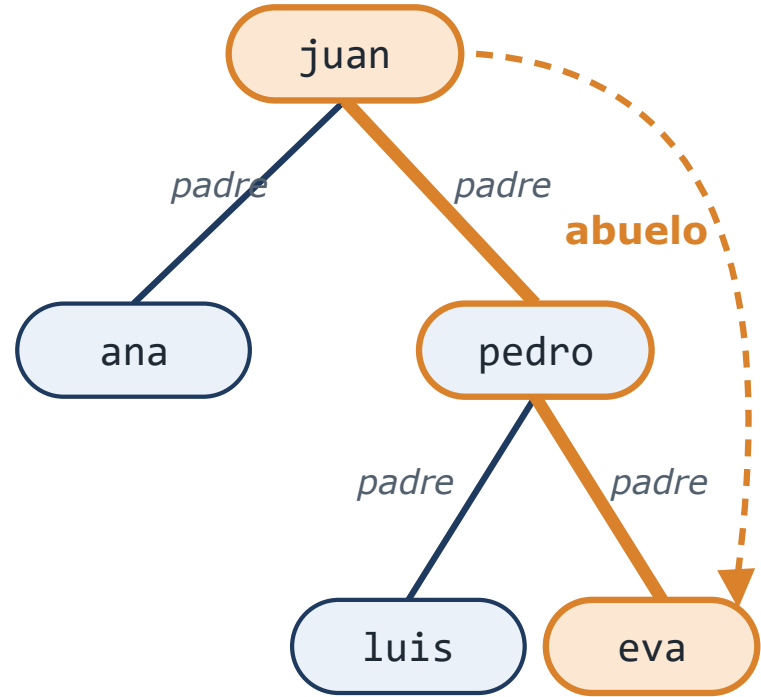
y

A es abuelo de N si A es padre de P y P es padre de N

Una deducción

```
abuelo(A, N) :-  
    padre(A, P),  
    padre(P, N).
```

```
?- abuelo(juan, eva).  
true.
```



Otras dos consultas

Sentido inverso

```
?- padre(Quien, ana).  
Quien = juan.
```

Variable anónima

```
?- padre(juan, _).  
true ;  
true.
```

¿Punto o punto y coma?

```
?- abuelo(juan, eva).  
true.
```

```
?- abuelo(juan, luis).  
true ;  
false.
```

Definición por casos

```
%! etapa(?P, ?E) is nondet.  
%  
% E es la etapa de la vida en la que está P, según su edad.  
etapa(P, bebe) :-  
    edad(P, A),  
    A < 4.  
etapa(P, chico) :-  
    edad(P, A),  
    A >= 4,  
    A < 18.  
etapa(P, adulto) :-  
    edad(P, A),  
    A >= 18.
```

Tres casos

```
?- etapa(sofia, Etapa).  
Etapa = bebe ;  
false.
```

```
?- etapa(juan, Etapa).  
Etapa = adulto.
```

orden de las cláusulas

bebe

cláusula 1

$A < 4$

sofia

chico

cláusula 2

$A \geq 4, A < 18$

luis, eva

adulto

cláusula 3

$A \geq 18$

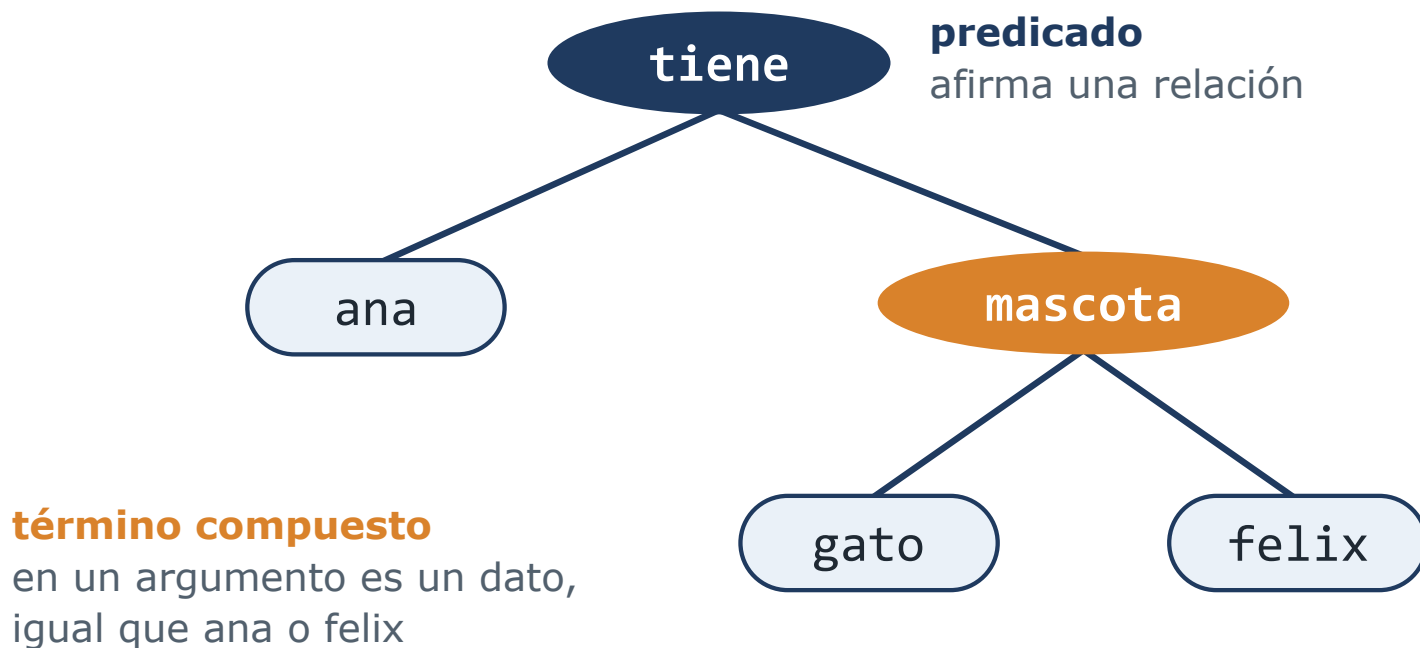
juan, ana, pedro

Términos compuestos

```
% tiene(P, M): P tiene la mascota M.  
tiene(ana, mascota(gato, felix)).  
tiene(luis, mascota(perro, rocco)).  
tiene(eva, mascota(gato, gaturro)).  
tiene(pedro, mascota(tortuga, manuelita)).  
  
?- tiene(ana, mascota(Especie, Nombre)).  
Especie = gato,  
Nombre = felix.
```

Un término es un árbol

`tiene(ana, mascota(gato, felix)).`



Un componente

```
%! propietario_de_gato(?P) is nondet.  
%  
% P tiene por lo menos un gato.  
propietario_de_gato(P) :-  
    tiene(P, mascota(gato, _)).  
  
?- propietario_de_gato(Quien).  
Quien = ana ;  
Quien = eva.
```

Unificación

```
?- ana = ana.  
true.
```

```
?- ana = pedro.  
false.
```

```
?- Quien = ana.  
Quien = ana.
```

El igual no evalúa

?- $X = 2 + 1.$

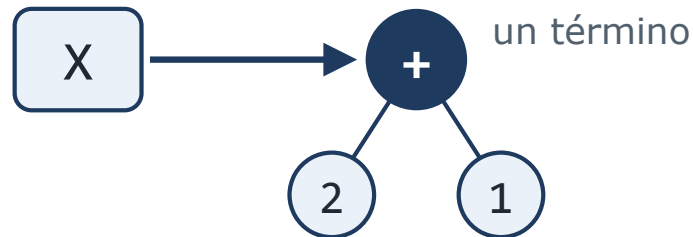
$X = 2+1.$

?- $2 + 1 = 3.$

false.

unificación

$X = 2 + 1$



evaluación

X is $2 + 1$



Comparar y negar

```
#!/ mayor_que(?A, ?B) is nondet.
```

```
%
```

```
% A tiene más años que B.
```

```
mayor_que(A, B) :-  
    edad(A, EdadA),  
    edad(B, EdadB),  
    EdadA > EdadB.
```

```
#!/ distintos(+A, +B) is semidet.
```

```
%
```

```
% A y B no son la misma persona.
```

```
distintos(A, B) :-  
    \+ A = B.
```

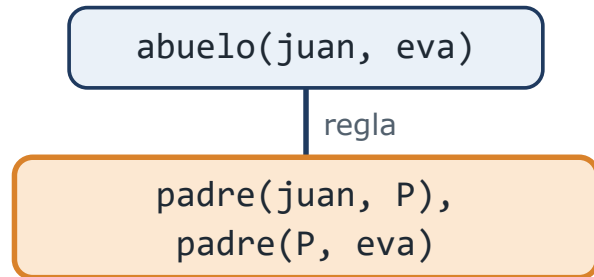
```
?- distintos(ana, pedro).  
true.
```

```
?- distintos(ana, ana).  
false.
```

La búsqueda · 1

```
padre(juan, ana).  
padre(juan, pedro).  
padre(pedro, luis).  
padre(pedro, eva).
```

```
abuelo(A, N) :-  
    padre(A, P),  
    padre(P, N).
```

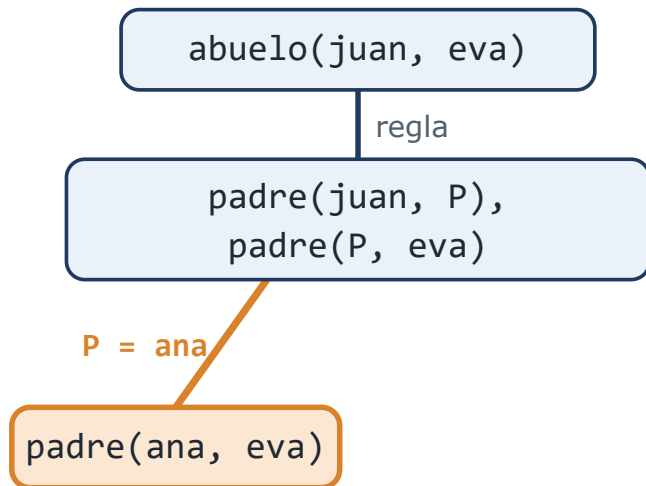


Call: (10) abuelo(juan, eva)

La búsqueda - 2

```
padre(juan, ana).  
padre(juan, pedro).  
padre(pedro, luis).  
padre(pedro, eva).
```

```
abuelo(A, N) :-  
    padre(A, P),  
    padre(P, N).
```

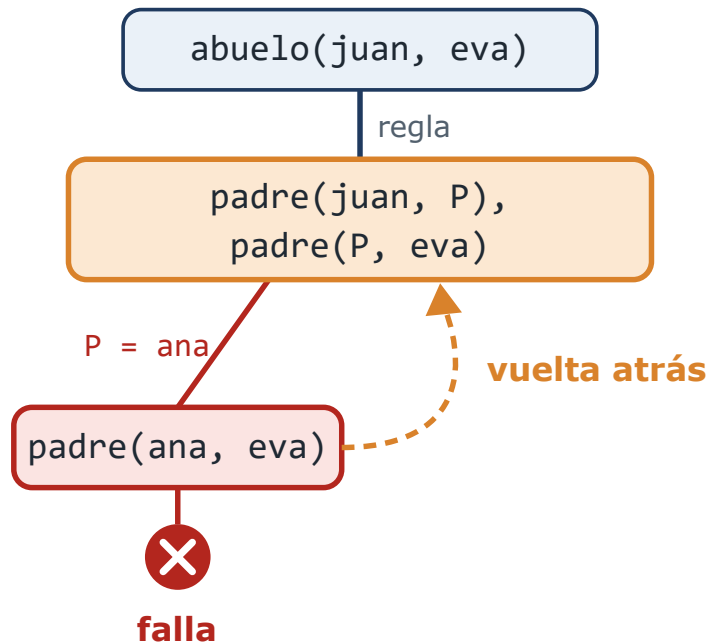


```
Call: (11) padre(juan, _)  
Exit: (11) padre(juan, ana)  
Call: (11) padre(ana, eva)
```

La búsqueda - 3

```
padre(juan, ana).  
padre(juan, pedro).  
padre(pedro, luis).  
padre(pedro, eva).
```

```
abuelo(A, N) :-  
    padre(A, P),  
    padre(P, N).
```

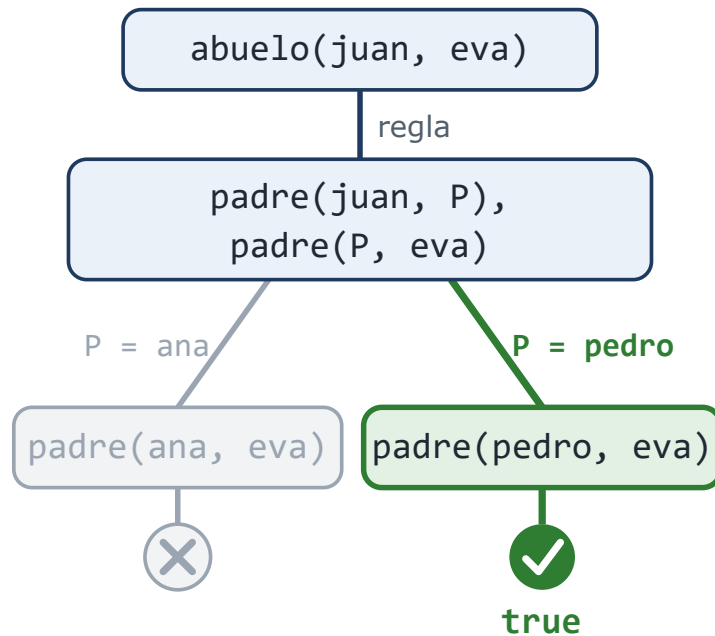


```
Fail: (11) padre(ana, eva)  
Redo: (11) padre(juan, _)
```

La búsqueda - 4

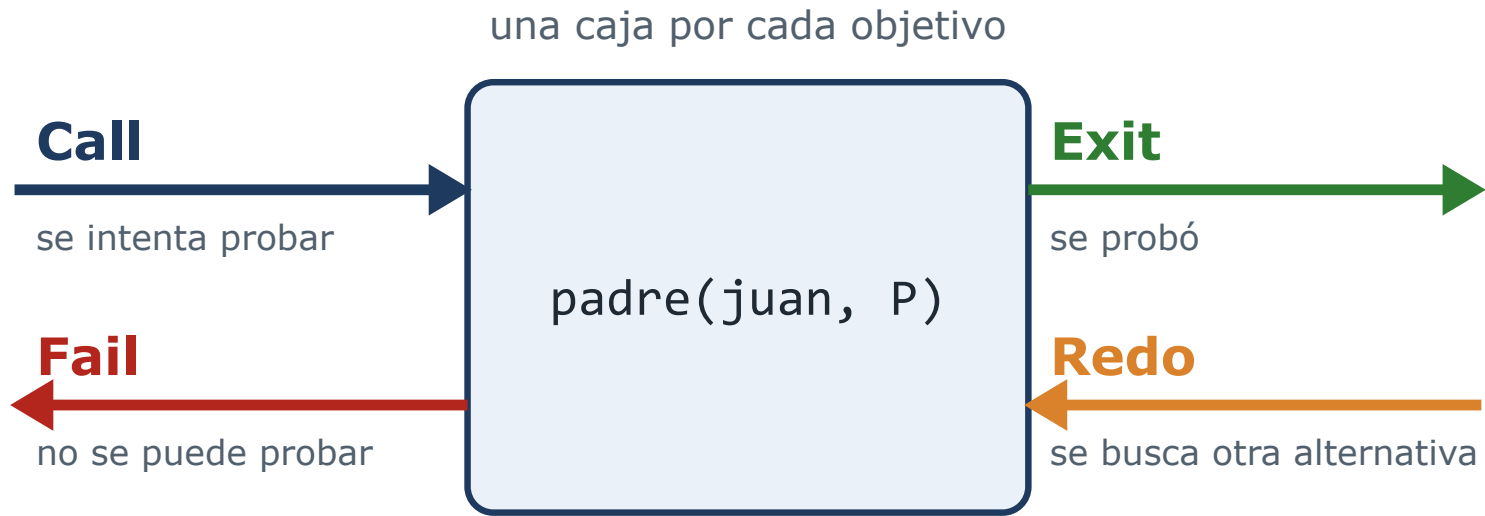
```
padre(juan, ana).  
padre(juan, pedro).  
padre(pedro, luis).  
padre(pedro, eva).
```

```
abuelo(A, N) :-  
    padre(A, P),  
    padre(P, N).
```

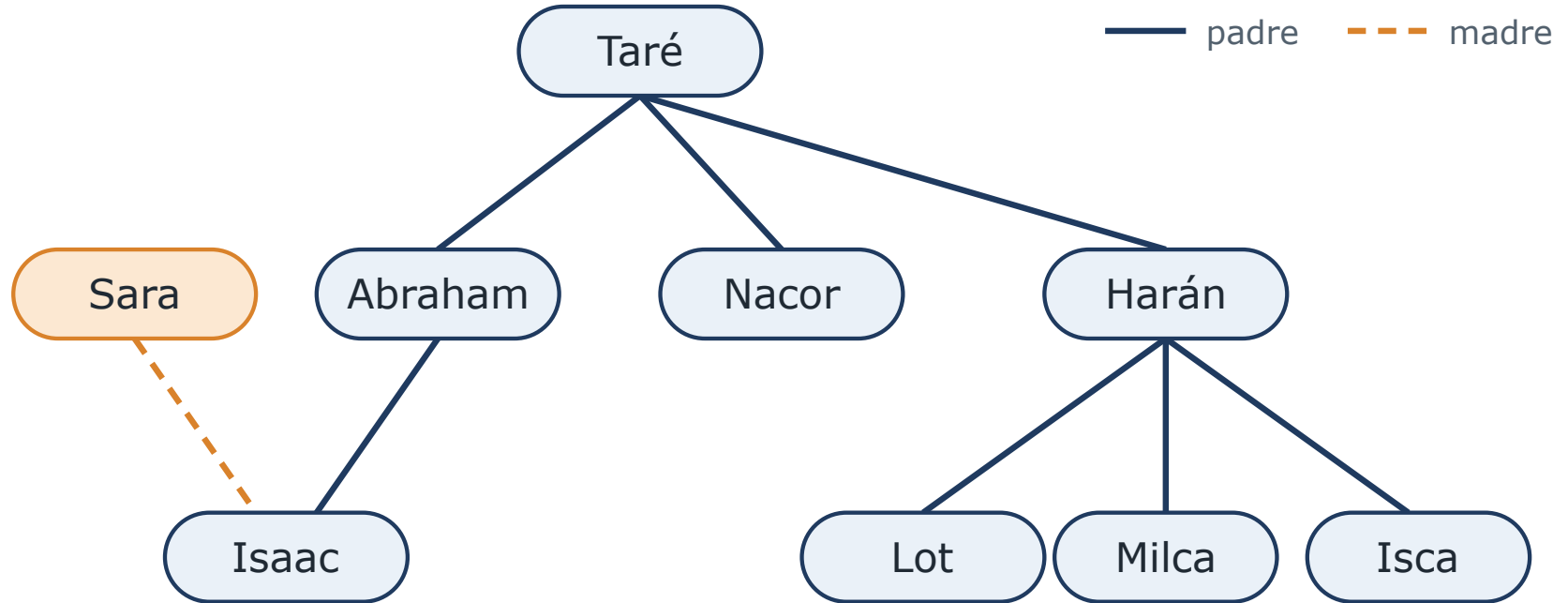


```
Exit: (11) padre(juan, pedro)  
Call: (11) padre(pedro, eva)  
Exit: (11) padre(pedro, eva)  
Exit: (10) abuelo(juan, eva)
```

Cuatro puertas



El árbol de Taré



Recursión

```
antepasado(A, D) :-  
    progenitor(A, D).  
antepasado(A, D) :-  
    progenitor(A, Hijo),  
    antepasado(Hijo, D).
```

antepasado(tare, isaac)

cláusula 1 (caso base)

progenitor(tare, isaac)

falla

cláusula 2 (caso recursivo)

progenitor(tare, Hijo)

Hijo = abraham

antepasado(abraham, isaac)

cláusula 1 (caso base)

progenitor(abraham, isaac)



se prueba: termina

true

Primero los hijos

```
?- antepasado(tare, Quien).
```

```
Quien = abraham ;
```

```
Quien = nacor ;
```

```
Quien = haran ;
```

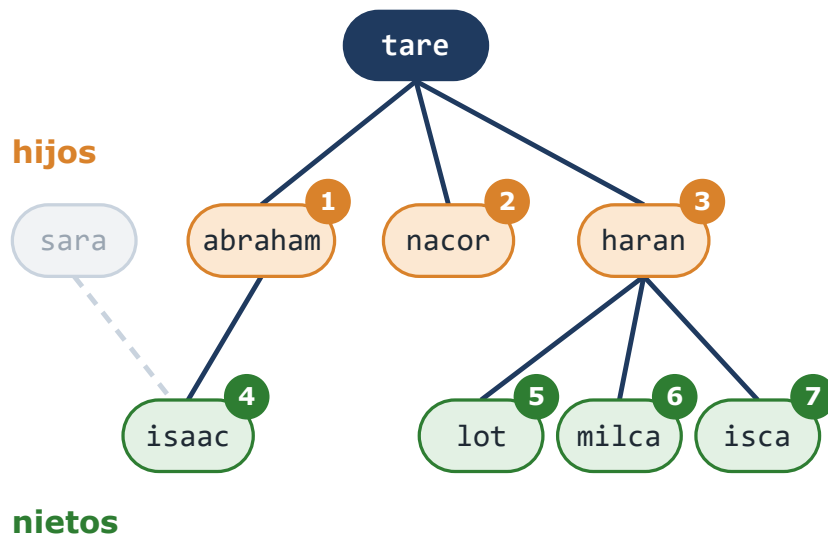
```
Quien = isaac ;
```

```
Quien = lot ;
```

```
Quien = milca ;
```

```
Quien = isca ;
```

```
false.
```



Aritmética

%! doble(+N, -D) is det.

%

% D es el doble de N.

`doble(N, D) :-`

`D is N * 2.`

`?- doble(21, X).`

`X = 42.`

`?- doble(X, 42).`

`ERROR: Arguments are not sufficiently instantiated`

Salida de texto

```
%! saludar(+A) is det.
```

```
%
```

```
% Escribe un saludo para A y pasa a la línea siguiente.
```

```
saludar(A) :-  
    write('Hola, '),  
    write(A),  
    nl.
```

```
?- saludar(ana).
```

```
Hola, ana
```

```
true.
```

format/2

```
presentar(A, N) :-  
    format("~w tiene ~w hermanos~n", [A, N]).
```

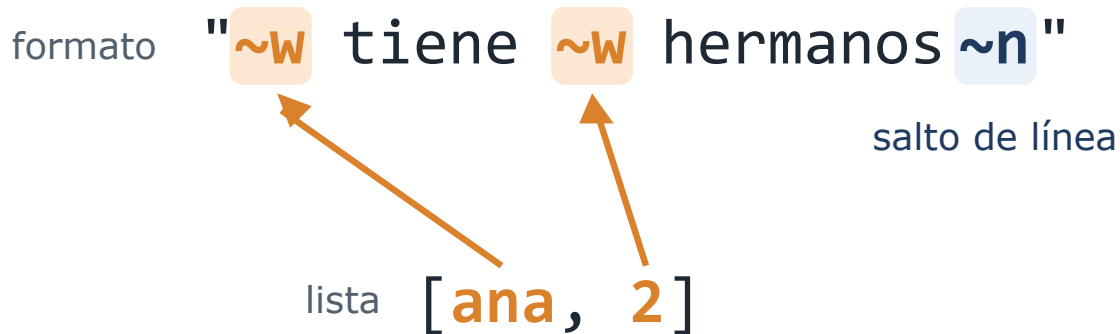
```
?- presentar(ana, 2).  
ana tiene 2 hermanos  
true.
```

Texto con valores

formato " ~W tiene ~W hermanos ~n "

salto de línea

lista [ana, 2]



ana tiene 2 hermanos

Contar de 1 a 5

```
?- between(1, 5, N).
```

```
N = 1 ;
```

```
N = 2 ;
```

```
N = 3 ;
```

```
N = 4 ;
```

```
N = 5.
```

```
?- cuenta(1, 5).
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

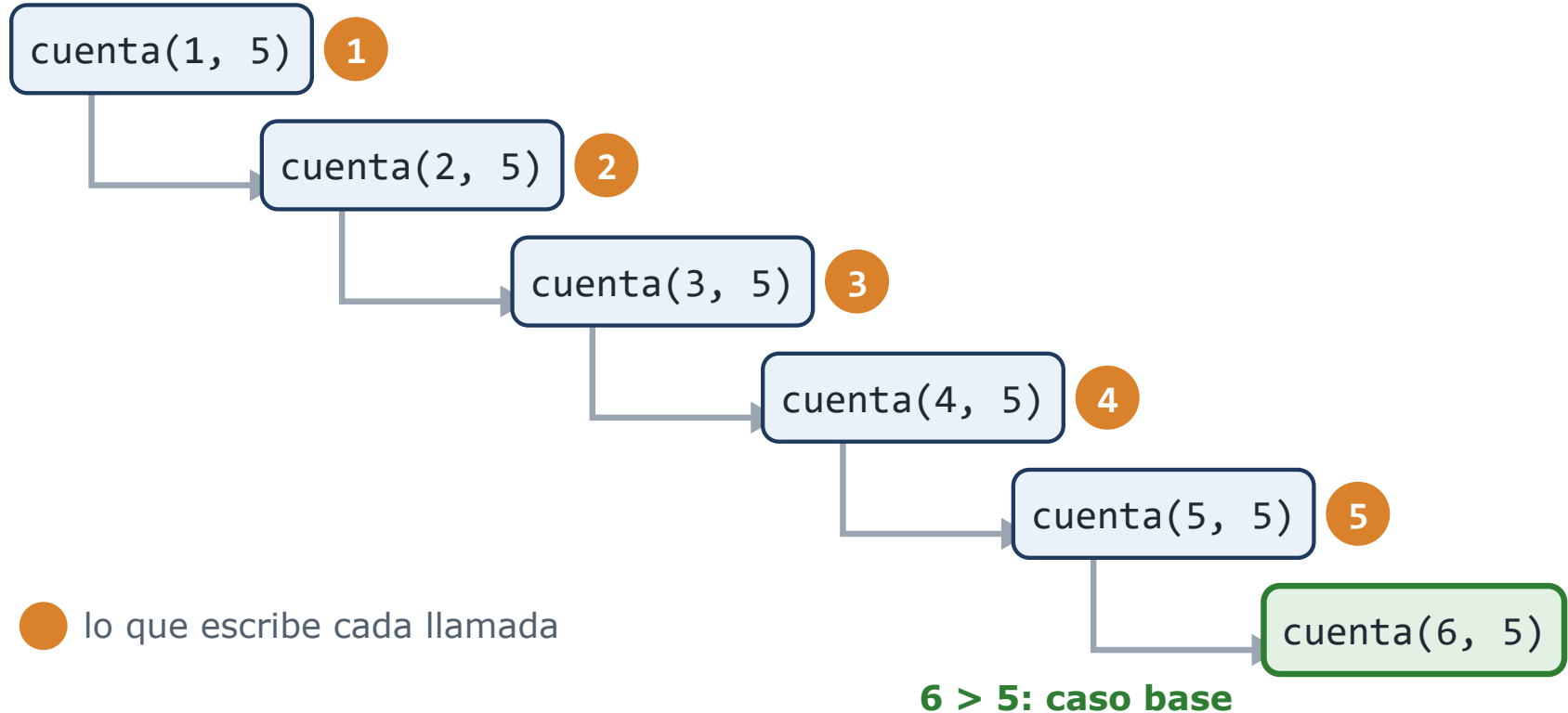
```
true ;
```

```
false.
```

cuenta/2

```
#!/ cuenta(+Desde, +Hasta) is det.  
%  
% Escribe Los números de Desde a Hasta, uno por Línea.  
cuenta(Desde, Hasta) :-  
    Desde =< Hasta,  
    format("~w~n", [Desde]),  
    Siguiente is Desde + 1,  
    cuenta(Siguiente, Hasta).  
cuenta(Desde, Hasta) :-  
    Desde > Hasta.
```

Una llamada por número



Listas

[ana, luis, eva, sofia]



Primero = ana

Resto = [luis, eva, sofia]

[Primero|Resto]

Pertenencia

```
invitados([ana, luis, eva, sofia]).
```

```
%! esta_invitado(?P) is nondet.
```

```
%
```

```
% P está en la lista de invitados.
```

```
esta_invitado(P) :-
```

```
    invitados(Lista),
```

```
    member(P, Lista).
```

```
?- esta_invitado(Quien).
```

```
Quien = ana ;
```

```
Quien = luis ;
```

```
Quien = eva ;
```

```
Quien = sofia.
```

[Primero|Resto]

```
%! primero_en_llegar(-P) is det.
```

```
%
```

```
% P es el primer elemento de la lista.
```

```
primero_en_llegar(P) :-  
    invitados([P|_]).
```

```
%! los_demas(-Resto) is det.
```

```
%
```

```
% Resto es la lista sin su primer elemento.
```

```
los_demas(Resto) :-  
    invitados([_|Resto]).
```

```
?- los_demas(Resto).
```

```
Resto = [luis, eva, sofia].
```

Resumen

Programa

:-	se lee «si»
,	se lee «y»
.	fin de cláusula
%	comentario

Consultas

?-	consulta
;	otra respuesta
_	variable anónima
Quien	variable

Términos

=	¿unifican?
\+	no se prueba
f(a, b)	compuesto

Números

is	evalúa
< =< > >=	compara
between/3	genera

Salida

write/1	un término
nl/0	salto de línea
format/2	~w ~n

Listas y traza

member/2	pertenencia
length/2	longitud
[P R]	primero y resto
trace/0	ver la búsqueda

Créditos

Retrato de Alain Colmerauer (1988): Alaindavid2, CC BY-SA 4.0, Wikimedia Commons, commons.wikimedia.org/wiki/File:A-Colmerauer_web-800x423.jpg

Ilustraciones: dibujadas para el curso, licencia MIT.

Ejemplos: ejemplos/capitulo-01 del curso.